CSE312 Software Architecture: Principles and Practices



Course Details

- Code: CSE312
- Title: Software Architecture: Principles and Practices
- L-T-P-C: 3-0-2-4
- Faculty: Anitha Ambat
- Office: AC318, Academic Block 1
- Email: anithaambat@iiitkottayam.ac.in

Syllabus

- Introduction to software engineering
- Software life cycle model
- Requirement analysis and specification
- Architecture and Design
- System Design
- Software testing
- Software Project Management

Evaluation Mode

- **Theory**
  - Assignment-1 — Weightage: 10
  - Quiz-1 — Weightage: 10
  - Mid Semester Examination — Weightage: 20
  - Quiz-2 — Weightage: 10
  - Assignment-2: — Weightage: 10
  - End-semester Exam — Weightage: 40
  - **Total** — **Weightage: 70**

- **Lab (Project Based)**
  - Project Presentation — 15
  - Progress — 10
  - Viva — 5
  - **Weightage** — **30**

Organization of this Lecture:

- What is Software Engineering?
- Programs vs. Software Products
- Evolution of Software Engineering
- Notable Changes In Software Development Practices
- Introduction to Life Cycle Models
- Summary
- **R. Mall, "Fundamentals of Software Engineering," Prentice-Hall of India, 1999, CHAPTER 1.**

Introduction to Software Engineering

- **Software** is more than just a program code.
- A program is an executable code, which serves some computational purpose.
- Software is considered to be collection of executable programming code, associated libraries and documentations.
- Software, when made for a specific requirement is called software product.
- Software Engineering provides a standard procedure to design and develop a software.

Software Engineering is an engineering branch related to the evolution of software product using well-defined scientific principles, techniques, and procedures.



What is Software Engineering?

- Engineering approach to develop software.
  - Building Construction Analogy.
- Systematic collection of past experience:
  - techniques,
  - Methodologies,
  - Guidelines.



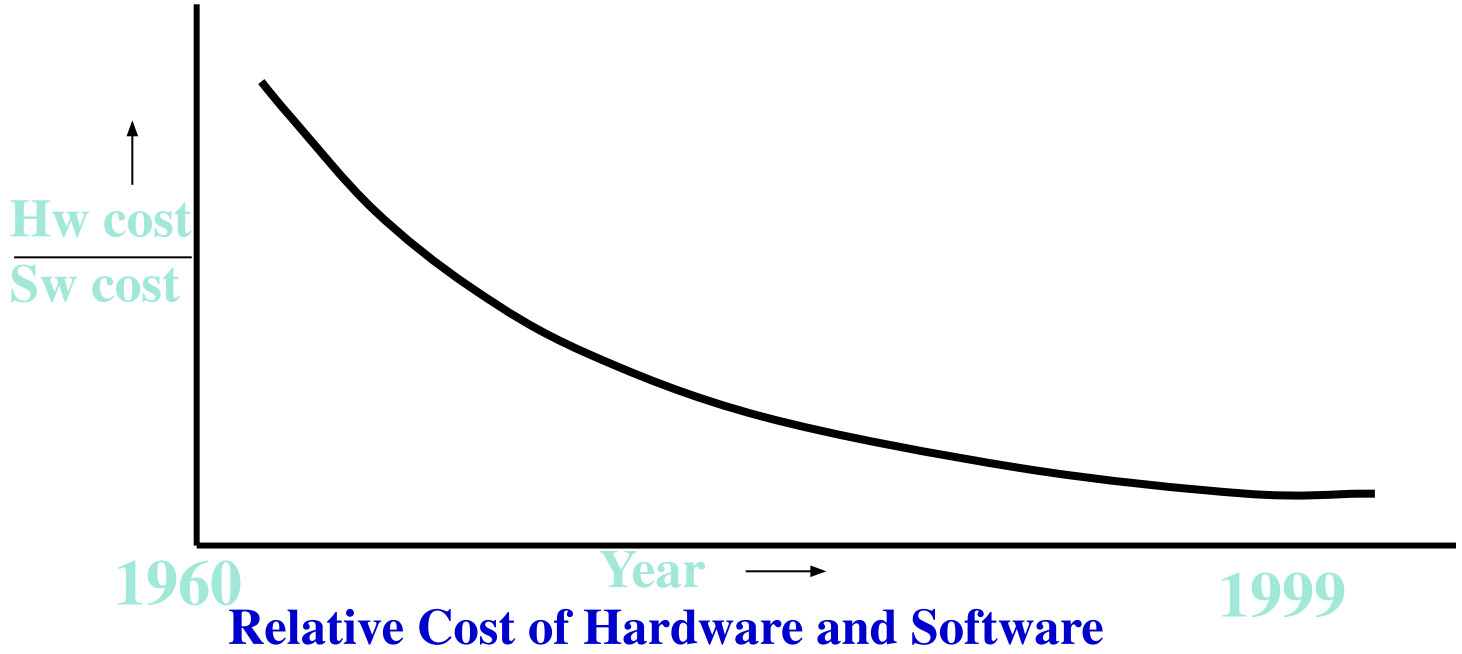
Why Study Software Engineering?

- To acquire skills to develop large programs.
  - Exponential growth in complexity and difficulty level with size.
- Ability to solve complex programming problems:
  - How to break large projects into smaller and manageable parts?
- Learn techniques of:
  - specification, design, interface development, testing, project management, etc
- To acquire skills to be a better programmer:
  - Higher Productivity
  - Better Quality Programs

Software Crisis

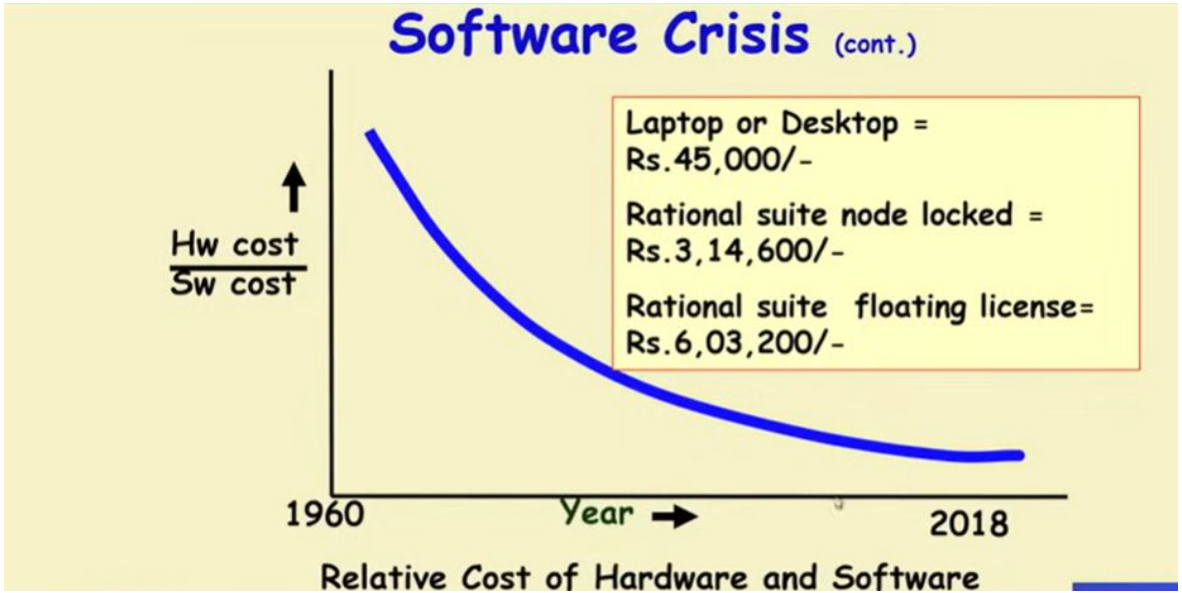
- Software products:
  - fail to meet user requirements.
  - frequently crash.
  - expensive.
  - difficult to alter, debug, and enhance.
  - often delivered late.
  - use resources non-optimally.

Software Crisis (cont.)



Relative Cost of Hardware and Software

Software Crisis (cont.)



Software costs are high compared to hardware.Why?

- **Manual Effort and Skill**: Unlike mass-produced hardware, creating software is a manual process that depends heavily on the knowledge and effort of the developers
- **Productivity Gap:** The complexity of the problems we solve with software is increasing rapidly, but the productivity of developers is not improving at the same speed
- **Project Delays:** Roughly half of all software projects are released late, and these delays cause the cost to rise far beyond the original budget
- **Complexity Management:** For large projects, the effort and time required can increase exponentially, especially when systematic engineering techniques are not used to manage the human brain's memory limitations

Factors contributing to the software crisis

- Larger problems,
- Lack of adequate training in software engineering,
- Increasing skill shortage,
- Low productivity improvements

Programs versus Software Products

Usually small in size	Large
Author himself is sole user	Large number of users
Single developer	Team of developers
Lacks proper user interface	Well-designed interface
Lacks proper documentation	Well documented & user-manual prepared
Ad hoc development.	Systematic development

Exploratory Program Development Style

- It is an informal way of developing software.
- The programmer depends on their own experience and intuition.
- There is no fixed process or standard methodology.
- The programmer has complete freedom to decide how to develop the software.
- Development usually starts after a brief discussion with the customer.
- The programmer starts coding immediately.
- After coding, the software is tested.
- Bugs are identified and fixed.
- The process of testing and bug fixing is repeated until the customer is satisfied.
- This approach is suitable for small and simple programs, but not for large software projects.

Differences between the exploratory style and modern software development practices

Feature	Exploratory Style (Build and Fix)	Modern Software Engineering
Core Approach	Relies on **intuition** and a "build and fix" method because no other systematic techniques were available.	Uses a **systematic engineering approach** where experiences are studied, documented, and based on fundamental techniques.
Scalability	Suitable only for small programs; it is not effective for large-scale projects.	Specifically designed to handle large and complex software projects.
Effort and Time	Effort and time increase exponentially as the project grows, eventually leading to a collapse where the project cannot be continued.	Effort remains manageable; while it may take more time for larger problems, it does not escalate to impossible levels

Differences between the exploratory style and modern software development practices

Feature	Exploratory Style (Build and Fix)	Modern Software Engineering Practices
Error Rate	Typically results in a high number of errors.	Focuses on techniques to manage complexity and improve reliability.
Team Collaboration	Difficult to use for team-based development.	Provides a framework that supports collaborative, team-oriented development.
Complexity Management	Does not provide tools to overcome human cognitive limitations, leading to breakdown when complexity exceeds memory capacity.	Employs techniques like Abstraction and Decomposition to help the human brain manage complexity by focusing on relevant details.

Human Cognitive limitation

- Complexity and cost increases exponentially if SE principles are not used.
- There are two fundamental techniques to carry out the complexity
  - Abstraction
  - Decomposition
- They effectively handle complexity as the program size increases

What is abstraction?

- Focus attention on one aspect of the problem and ignore irrelevant details.
  - Also known as model building.
- **Simplify a problem by omitting unnecessary details.**
- Eg-To develop overall understanding of a country.
- Will refer various types of maps.
- A map is an abstract representation of a country.
- Different types of maps(abstractions) are possible.
- Several abstraction to the same problem can be created

Does every problem have a single Abstraction?

- Different types of models help to understand different aspects of the problem.
- For Complex problems
- A single level of abstraction may be inadequate.
- A hierarchy of abstractions may have to be constructed

Decomposition

- Decompose a problem into many small independent parts.
- The smaller parts are taken up one by one and solved separately.
- **Idea- Each small part would be easy to grasp and therefore can be easily solved.**
- The full problem is solved when all the parts are solved.
- Eg. 1) Try to break a bunch of sticks tied together versus breaking them individually.
- An arbitrary decomposition may not help.
- 2) You understand a book easier when the contents are organized into independent chapters.

Types of Software Development Projects

- Software Products
  - Developed for many users
  - Developed once, sold many times
  - Same software used by different customers (Generic software product)
  - Different versions for different users
  - Generates continuous revenue
  - Regular updates and maintenance
  - Examples - Microsoft Windows,Microsoft Office,Google Chrome,Adobe Photoshop

Types of Software Development Projects

- Software Services
  - Developed for a specific customer
  - Based on customer requirements
  - Usually customized from existing software
  - Faster and cost-effective
  - Generates one-time revenue
  - Examples - College Management System, Hospital Management System, Library Management System

Types of Software Development Projects

- Customized Software
  - Developed for one organization
  - Existing software is reused
  - Only required features are modified
  - Saves development time and cost
  - Example:- College ERP developed for a university

Types of Software Development Projects

- Outsourced Software
  - Part of a project is given to another company
  - Used when special expertise is needed
  - Reduces development time
  - Improves efficiency
  - Example: - TCS outsources the mobile app module to another company.

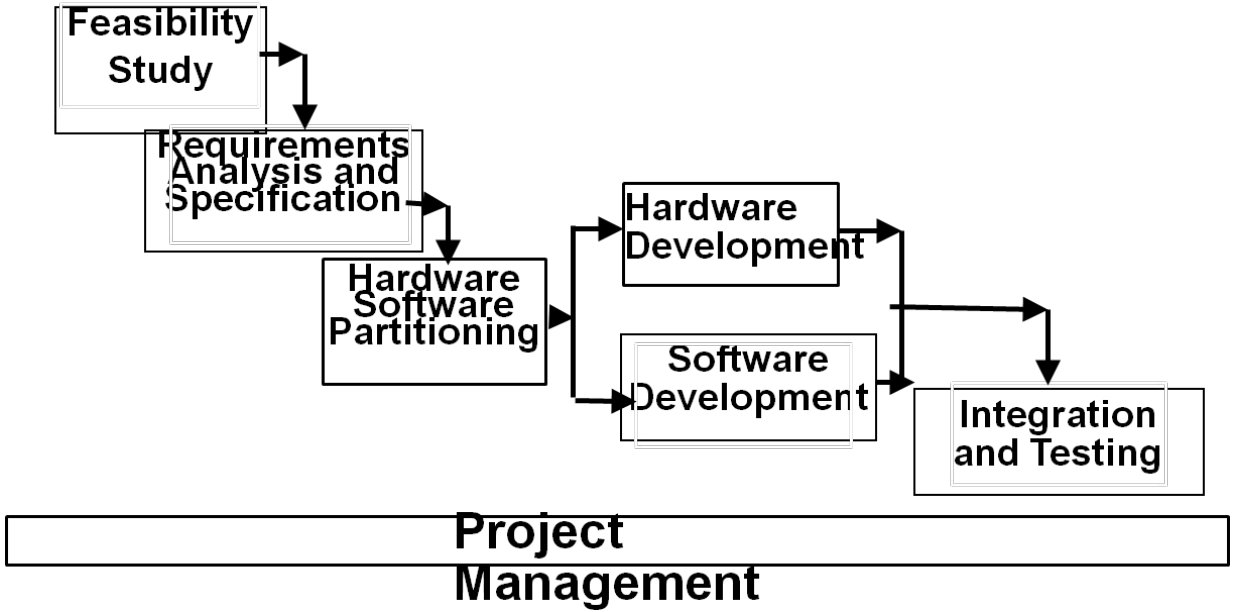
Computer Systems Engineering

- Computer systems engineering:
  - encompasses software engineering.
- Many products require development of software as well as specific hardware to run it:
  - a coffee vending machine,
  - a mobile communication product, etc
- The high-level problem:
  - deciding which tasks are to be solved by software
  - which ones by hardware

Computer Systems Engineering

- Often, hardware and software are developed together:
  - Hardware simulator is used during software development.
- Integration of hardware and software.
- Final system testing

Computer Systems Engineering



Emergence of Software Engineering

- **Early Computer Programming (1950s):**
  - Programs were being written in assembly language.
  - Programs were limited to about a few hundreds of lines of assembly code.
  - Every programmer developed his own style of writing programs:
    - according to his intuition (exploratory programming)
- **High-Level Language Programming (Early 60s)**
  - High-level languages such as FORTRAN, ALGOL, and COBOL were introduced:
    - This reduced software development efforts greatly.
  - Software development style was still exploratory.
    - Typical program sizes were limited to a few thousands of lines of source code.

Emergence of Software Engineering

- **Control Flow-Based Design (late 60s)**
  - Size and complexity of programs increased further
    - exploratory programming style proved to be insufficient
  - Programmers found:
    - very difficult to write cost-effective and correct programs.
    - programs written by others very difficult to understand and maintain
  - To cope up with this problem, experienced programmers advised: “Pay particular attention to the design of the program's control structure.”
  - A program's control structure indicates:
    - the sequence in which the program's instructions are executed.
  - To help design programs having good control structure:
    - flow charting technique was developed.

Emergence of Software Engineering

- Using flowcharting technique:
  - one can represent and design a program's control structure.
  - Usually one understands a program:
    - by mentally simulating the program's execution sequence.
- A program having a messy flow chart representation:
  - difficult to understand and debug.
- It was found:
  - GO TO statements makes control structure of a program messy
  - GO TO statements alter the flow of control arbitrarily.
  - The need to restrict use of GO TO statements was recognized.
- Many programmers had extensively used assembly languages.
  - JUMP instructions are frequently used for program branching in assembly languages,
  - programmers considered use of GO TO statements inevitable.

Emergence of Software Engineering

- At that time, Dijkstra published his article:
  - “Goto Statement Considered Harmful” Comm. of ACM, 1969.
- Many programmers were unhappy to read his article
- They published several counter articles:
  - highlighting the advantages and inevitability of GO TO statements.
- But, soon it was conclusively proved:
  - only three programming constructs are sufficient to express any programming logic:
  - sequence (e.g. a=0;b=5;)
  - selection (e.g.if(c=true) k=5 else m=5;)
  - iteration (e.g. while(k>0) k=j-k;)
- Everyone accepted:
  - it is possible to solve any programming problem without using GO TO statements.
  - This formed the basis of Structured Programming methodology.

Emergence of Software Engineering

- **Structured Programming**
  - A program is called structured
    - when it uses only the following types of constructs:
    - sequence, selection, iteration
  - Unstructured control flows are avoided.
  - Consist of a neat set of modules.
  - Use single-entry, single-exit program constructs.
  - However, violations to this feature are permitted:
    - due to practical considerations such as:
      - premature loop exit (break) to support exception handling

Emergence of Software Engineering

- Structured programs are:
  - Easier to read and understand,
  - easier to maintain,
  - require less effort and time for development.
- Research experience shows:
  - programmers commit less number of errors
    - while using structured **if-then-else** and **do-while** statements
    - compared to test-and-branch (GOTO)constructs.

Emergence of Software Engineering

- Data Structure-Oriented Design (Early 70s)
  - Soon it was discovered:
    - it is important to pay more attention to the design of data structures of a program
      - than to the design of its control structure
  - Techniques which emphasize designing the data structure:
    - derive program structure from it:
      - are called data structure-oriented design techniques.
  - Example of data structure-oriented design technique:
    - Jackson's Structured Programming(JSP) methodology
      - developed by Michael Jackson in 1970s.

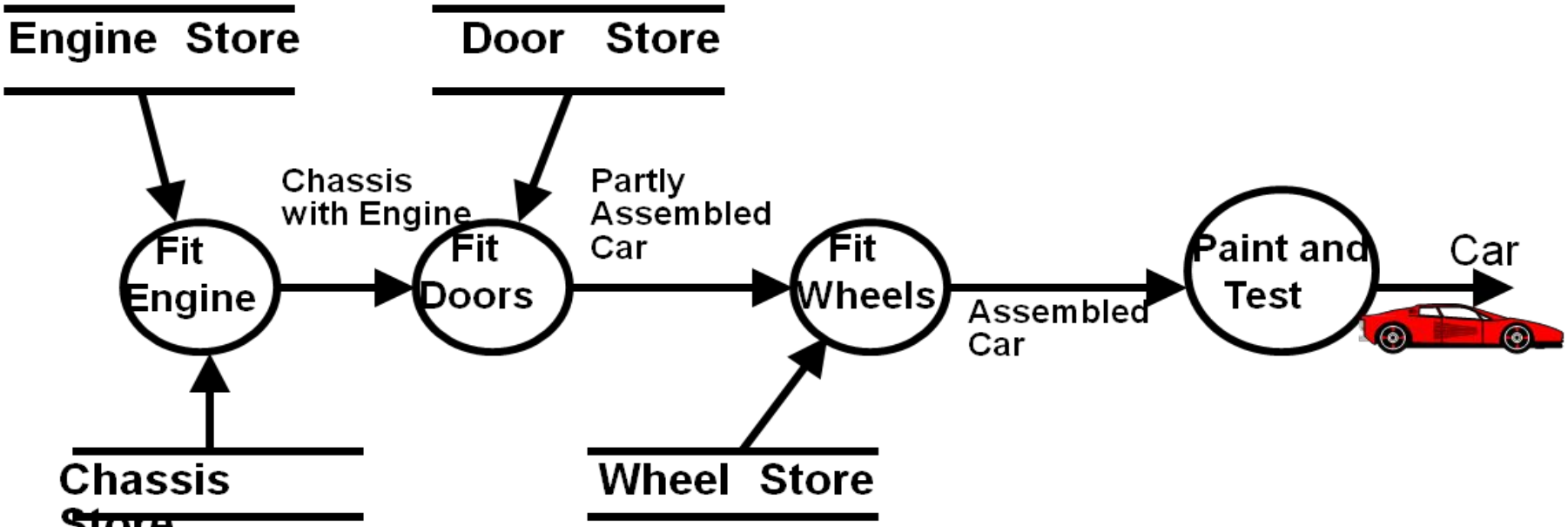
Emergence of Software Engineering

- JSP technique:
  - program code structure should correspond to the data structure.
- In JSP methodology:
  - a program's data structures are first designed using notations for
    - sequence, selection, and iteration.
  - Then data structure design is used :
    - to derive the program structure.
- Several other data structure-oriented Methodologies also exist:
  - e.g., Warnier-Orr Methodology.

Emergence of Software Engineering

- **Data Flow-Oriented Design (Late 70s)**
  - Data flow-oriented techniques advocate:
    - the data items input to a system must first be identified,
    - processing required on the data items to produce the required outputs should be determined
- Data flow technique identifies:
  - different processing stations (functions) in a system
  - the items (data) that flow between processing stations
- Data flow technique is a generic technique:
  - can be used to model the working of any system
    - not just software systems.
- A major advantage of the data flow technique is its simplicity.

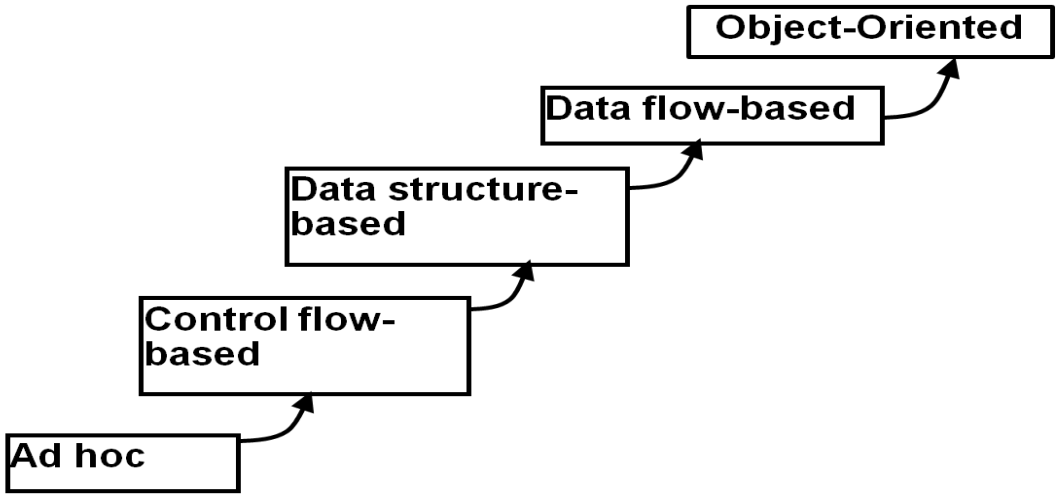
Data Flow Model of a Car Assembly Unit



Emergence of Software Engineering

- **Object-Oriented Design (80s)**
  - an intuitively appealing design approach:
  - natural objects (such as employees, pay-roll-register, etc.) occurring in a problem are first identified.
- Relationships among objects:
  - such as composition, reference, and inheritance are determined.
- Each object essentially acts as
  - a data hiding (or data abstraction) entity.
- Object-Oriented Techniques have gained wide acceptance:
  - Simplicity,Reuse possibilities,Lower development time and cost,More robust code,Easy maintenance

Evolution of Design Techniques



Evolution of Other Software Engineering Techniques

- The improvements to the software design methodologies
  - are indeed very conspicuous.
- In additions to the software design techniques:
  - several other techniques evolved

Evolution of Other Software Engineering Techniques

- life cycle models,
- specification techniques,
- project management techniques,
- testing techniques,
- debugging techniques,
- quality assurance techniques,
- software measurement techniques,
- CASE(computer aided software engineering)tools, etc.

Software Life Cycle

- Software life cycle (or software process):
  - series of identifiable stages that a software product undergoes during its life time:
    - Feasibility study
    - requirements analysis and specification,
    - design,
    - coding,
    - Testing
    - maintenance.

Software Life Cycle (Cont.)

- A software life cycle model (or process model):
  - a descriptive and diagrammatic model of software life cycle
  - identifies all the activities required for product development,
  - establishes a precedence ordering among the different activities,
  - Divides life cycle into phases.
- Several different activities may be carried out in each life cycle phase.
  - For example, the design stage might consist of:
    - structured analysis activity followed by
    - structured design activity.

Why Model Life Cycle ?

- A written description:
  - forms a common understanding of activities among the software developers.
  - helps in identifying inconsistencies, redundancies, and omissions in the development process.
  - Helps in tailoring a process model for specific projects.
- Processes are tailored for special projects.
  - A documented process model
    - helps to identify where the tailoring is to occur.

Life Cycle Model (Cont.)

- The development team must identify a suitable life cycle model:
  - and then adhere to it.
  - Primary advantage of adhering to a life cycle model:
    - helps development of software in a systematic and disciplined manner.
- When a program is developed by a single programmer ---
  - he has the freedom to decide his exact steps.

Life Cycle Model (Cont.)

- When a software product is being developed by a team:
  - there must be a precise understanding among team members as to when to do what,
  - otherwise it would lead to chaos and project failure
- A software project will never succeed if:
  - one engineer starts writing code,
  - another concentrates on writing the test document first,
  - yet another engineer first defines the file structure
  - another defines the I/O for his portion first.

Life Cycle Model (Cont.)

- A life cycle model:
  - defines entry and exit criteria for every phase.
  - A phase is considered to be complete:
    - only when all its exit criteria are satisfied.
- The phase exit criteria for the software requirements specification phase:
  - Software Requirements Specification (SRS) document is complete, reviewed, and approved by the customer.
- A phase can start:
  - only if its phase-entry criteria have been satisfied.

Life Cycle Model (Cont.)

- It becomes easier for software project managers:
  - to monitor the progress of the project.
- When a life cycle model is adhered to,
  - the project manager can at any time fairly accurately tell,
    - at which stage (e.g., design, code, test, etc.) of the project is.
- Otherwise, it becomes very difficult to track the progress of the project
  - the project manager would have to depend on the guesses of the team members.
- This usually leads to a problem:
  - known as the 99% complete syndrome.

Life Cycle Model (Cont.)

- Many life cycle models have been proposed.
- We will confine our attention to a few important and commonly used models.
  - classical waterfall model
  - iterative waterfall,
  - evolutionary,
  - prototyping, and
  - spiral model



Thank you..